



家規 (House Rules) 完整使用指南

給電腦小白看的版本：你公司的「規矩」AI 幫你管 適用 erpilot v3.25+ 預計閱讀時間：15 分鐘上手



「家規」是什麼？為什麼你需要它？

像每個家庭都有家規（不能 11 點後吃零食、回家先洗手），每家公司也有自己的「業務規矩」：

- 採購超 NT\$10 萬 → 老闆要批
- 工單沒有「做法 (Recipe)」→ 不能釋放到產線
- 銷售折扣超 5% → 主管要看
- 客戶信用額度爆了 → 不准開單
- 出貨前發票必須開好

傳統作法：這些規矩寫進員工腦袋裡。問題是：

- 🤔 新人不知道
- 🤔 老人忘記
- 🤔 換手沒交接
- 🤔 老闆不在大家偷偷放行
- 🤔 出事後找不到誰負責

erpilot 的「家規」把規矩寫進系統，這樣：

- ✅ 員工照規矩走，犯錯機率降低 80%
- ✅ 老闆睡得著（系統自動擋住違規）
- ✅ 換人不用重訓（規矩在系統裡）
- ✅ 主管想 emergency 放行 → 可覆寫，留 audit log
- ✅ ISO / GMP 稽核 → 一鍵匯出完整紀錄

VS 對手是怎麼做的？為什麼 erpilot 不一樣

ERP	怎麼設家規	缺點
SAP B1	顧問改 code	等 1 個月 + 花 5-20 萬
鼎新	設定畫面找半天	條件死板，特殊情境不支援
NetSuite	寫 JavaScript (SuiteScript)	小白做不到
Odoo	Python 表達式	危險 + 要會寫
erpilot ✨	👉 3 種任選	任何人都能改、立刻生效

🚀 erpilot 設家規的 3 種方式

方式 A：UI 點一點 (最簡單，給小白)

1. 登入 erpilot
2. 點左側「 審批」(v3.25 起家規與審批共用此頁)
3. 切「 規則設定」tab
4. 點「 新增規矩」
5. 填表單：

名稱：我家不准 PO 超 5 萬

觸發點：[PO 建立 ▼]

條件類型：[比較欄位 field_compare ▼]

欄位：amount (金額)

運算子：<= 小於等於

值：50000

動作：[擋住 block ▼]

訊息：金額太大，請拆單

可覆寫：[主管 manager ▼]

啟用：☒

[✓ 儲存] [X 取消]

6. 儲存 → 立即生效 (不用重啟、不用 deploy)

方式 B：對 AI 講話 (自然語言，erpilot 招牌)

不會用表單？用講的就好：

1. 點左上「 AI 助手」
2. 打字：

「我們公司 SO 折扣超過 5% 應該主管審」

3. AI 回應 (ConfirmCard 確認卡)：

 我幫您設定家規：

名稱：	SO 折扣 > 5% 需主管審
觸發：	銷售單建立 (so.create)
條件：	discount_pct > 0.05
動作：	要審批 (require_approval)
覆寫：	主管 (manager)

[✓ 確認] [X 取消]

4. 按 [✓ 確認] → 規則立即上線
5. 業務下次建超 5% 折扣的 SO → 自動進審批

 這就是 erpilot 對話式 ERP 最 powerful 的地方：用人話描述需求，AI 翻成系統規則。不會 SQL / 不會 JavaScript / 不會 Python 都能做。

方式 C：API / Plugin (給工程師客製化)

需要特殊條件 (例：自定義信用額度演算法)？工程師可以 plug-in 新的 condition 類型：

```
from app.services.policy_engine import register_condition

async def check_credit_limit(params, context, db):
    """檢查客戶信用額度 (plugin example) """
    from app.models.crm_sales import Customer
    from sqlalchemy import select
    cust_id = context.get('customer_id')
    if not cust_id:
        return False
    cust = (await db.execute(
        select(Customer).where(Customer.id == cust_id)
    )).scalar_one_or_none()
    if not cust:
        return False
    return context.get('amount', 0) <= cust.credit_limit

# 註冊新 condition 類型
register_condition("credit_check", check_credit_limit)
```

之後使用者就能在 UI 看到「信用額度檢查」作為一個 condition 選項。

家規的 4 個積木

1 觸發點 (Trigger)

「什麼動作會觸發這條規矩？」erpilot 內建 16+ 個觸發點：

類別	觸發點	何時觸發
生產	<code>wo.release</code> / <code>wo.complete</code> / <code>wo.cancel</code>	工單釋放 / 完工 / 取消
採購	<code>po.create</code> / <code>po.approve</code> / <code>po.receive</code> / <code>po.cancel</code>	採購單動作
銷售	<code>so.create</code> / <code>so.confirm</code> / <code>so.ship</code> / <code>so.cancel</code>	銷售單動作
庫存	<code>inventory.delete</code> / <code>inventory.transfer</code>	刪料件 / 調撥
會計	<code>journal.post</code> / <code>ar.create</code> / <code>ar.collect</code>	過帳 / 開發票 / 收款
CRM	<code>lead.convert</code> / <code>opportunity.stage_changed</code>	新苗轉客戶 / 追單推進

2 條件 (Condition)

「什麼情況才適用？」5 種內建類型：

條件類型	用途	範例
always	總是觸發 (沒例外)	「 PO 一律要記備註 」
has_bom	有做法才通過	「 WO 釋放需有 Recipe 」
field_compare	比較欄位	「 amount > 10 萬 」
count_check	計算列表	「 PO 至少 1 項目 」
custom	自定 (plugin 用)	「 信用額度檢查 」

field_compare 的運算子： `gt` / `gte` / `lt` / `lte` / `eq` / `ne`

3 動作 (Action)

「違反規矩怎麼處理？」4 種：

動作	行為	適用場景
 allow	純粹放行 (記 audit log)	內部統計用
 warn	不擋，UI 跳訊息提醒	例：「金額大，請確認」
 block	必須符合條件才能繼續，可覆寫	例：「需有 Recipe」
 require_approval	進審批流，需指定人點批准	例：「 PO > 10 萬 」

4 覆寫角色 (Override Role)

「誰可以放行被擋的動作？」常見值：

- `manager` — 主管
- `admin` — 系統管理員
- `null` — 沒人可覆寫 (最嚴格)

被擋時 UI 會跳「🔒 主管覆寫」按鈕，需指定角色的人按 + 填理由才能放行。



預設家規 (裝完系統自動有)

erpilot 開箱就送你 3 條最常用的：

- WO 釋放需有「做法 (Recipe)」
觸發: wo.release
條件: has_bom
動作: block
覆寫: manager
訊息: 「此產品還沒設定做法 (Recipe)。請先去生產頁 → 編做法。或請廠長覆寫。」
- PO > NT\$10 萬需主管審
觸發: po.create
條件: amount > 100000
動作: require_approval
覆寫: manager
- PO 必須至少 1 個項目
觸發: po.create
條件: items >= 1
動作: block
覆寫: 無

不要這些規矩？直接在 UI 關掉 is_active 就好。完全不用改 code、不用 deploy、不用顧問。

真實情境：阿玲（採購）的一天

場景：建一張高額採購單

1. 9:00 阿玲打開 erpilot「採購」頁
2. 點「+ 快速建單」
3. 填：供應商 = 長江廠 / 料件 = M6 螺絲 / 數量 = 1000 / 單價 = 150
4. 儲存 → 系統算總額 = NT\$ 150,000
5. 9:01 系統自動跑家規 evaluate：
 - 規矩「PO > 10 萬需主管審」命中 (150,000 > 100,000)
 - 動作 = require_approval
6. 阿玲看到：

「 PO-2026-0042 已建為草稿，待主管審批」

7. 9:05 主管林廠長手機收到通知 (v3.3 桌機 toast / Email digest)
8. 林廠長打開「 審批」頁 → 看到這張 PO 在「待我審」
9. 點「✓ 批准」→ 填一句註解「OK，這家信用好」→ 確認
10. PO 立即變成 approved，阿玲可以繼續走進貨流程

全程：

- ☒ 零寫死 code
- ☒ 零顧問費用
- ☒ 零教育訓練
- ☒ 完整 audit log (誰批的 / 何時 / 註解)

場景：緊急情況需要覆寫

某天客戶趕貨，但產品還沒設「做法 (Recipe)」：

1. 廠長嘗試 release WO → 被擋
2. 系統訊息：「此產品還沒設做法。請先去生產頁 → 編做法。或請廠長覆寫。」
3. 廠長按「 主管覆寫」
4. 跳輸入框：「請填覆寫原因 (會留稽核紀錄)」
5. 廠長填：「客戶趕貨，先放行，明天補做法」
6. 確認 → WO 釋放成功 + audit log 記「廠長 X 覆寫了規矩 Y，理由：客戶趕貨」

合規完整、彈性也夠。

稽核 log (給合規 / debug)

每次規矩 evaluate 都自動寫 `PolicyAuditLog`：

- 哪條規矩被觸發
- 評估結果 (`allowed` / `blocked` / `overridden` / `warned`)
- 上下文 (PO 金額 / 客戶 ID 等敏感資料自動截斷)
- 誰觸發 / 何時
- 主管覆寫者 + 理由 (如有)

ISO 9001 / GMP / 食安 / FDA 等合規場景需要這個 trail。查詢：`GET /api/policies/audit?rule_id=xxx&trigger=wo.release`

常見問題 FAQ

Q1: 我可以為不同 tenant (公司) 設不同家規嗎？

☒ 可以。 `PolicyRule` 含 `tenant_id`，多廠 / 多公司天然隔離。同一系統可服務多家公司，各有自己的規矩。

Q2: 規矩衝突怎麼辦？

同一 trigger 有多條規矩時，按 **priority** 升冚評估，第一條 **block** 的勝。建議命名約定：`priority < 50` 留給系統規則，`50-200` 給日常規矩，`200+` 給特殊情境。

Q3: 我能不能寫複雜的 Python 條件？

不建議直接 `eval` Python 字串（不安全）。請走 `register_condition()` plugin 機制：寫成 Python function + 命名 + 註冊 → UI 自動出現。

Q4: 規矩可以撤回嗎？

✓ 可以兩種方式：

- `is_active=false` 立即停用（推薦，保留歷史）
- DELETE 刪除（audit log 仍保留）

Q5: 主管覆寫太多會不會被濫用？

每次覆寫都寫 audit log，定期 review 即可。也可以加更嚴的 **override** 規則：

- 例：覆寫前要先 email 老闆
- 例：覆寫額度上限（一個月最多 5 次）

Q6: 規矩會不會擋住系統自己的後台流程？

不會。erpilot 設計上只有 **user-initiated** 動作會跑家規，系統 cron / cleanup / migration 不觸發。

Q7: 我看不懂英文 trigger 名（po.create / so.confirm）...

未來會加 i18n 翻譯。目前列表有中文描述。AI 助手對話建規矩時，你只要講人話，AI 會幫你翻 trigger 名。

Q8: 我能不能匯出 / 匯入家規（公司搬家 / 部署多廠）？

可以用 `GET /api/policies/rules` 拉 JSON，`POST` 灌進新環境。完整 import/export UI 是 Phase 2。

🔧 給工程師：技術細節

Schema

```
class PolicyRule(Base, TenantMixin):
    id: str (UUID PK)
    name: str (max 200)
    description: text
    trigger: str (16+ 白名單)
    condition_type: str (5 內建 / plugin)
    condition_params: JSON
    action: str (allow/warn/block/require_approval)
    message: str
    override_role: str | None
    is_active: bool (default True)
    priority: int (default 100)
    created_by / created_at / updated_at

class PolicyAuditLog(Base, TenantMixin):
    id, rule_id, trigger, action_taken
    context: JSON (auto-truncated)
    user_id, override_by, override_reason
    created_at
```

整合到 service

```
# 之前 (v3.24 寫死)
if not bom:
    raise BusinessRuleError("需 BOM")

# 之後 (v3.25 資料化)
result = await evaluate_policies(db, "wo.release", {"product_id": ...})
if result.blocked:
    raise BusinessRuleError(
        result.message,
        can_override=result.can_override,
        override_role=result.override_role,
    )
```

Plugin 新 condition

```
# 在 startup 或 plugin module
from app.services.policy_engine import register_condition

async def my_custom(params: dict, context: dict, db: AsyncSession) -> bool:
    # 回 True 表示「條件成立，規則放行」
    # 回 False 表示「條件不成立，觸發 action」
    return ...

register_condition("my_custom", my_custom)
```

相關文件

- [USER_MANUAL_ZH.md](#) — 完整使用者操作手冊
- [HOW_TO_GET_LLM_API_KEY_ZH.md](#) — 啟用 AI 對話
- 英文版：[HOUSE_RULES_GUIDE_EN.md](#)

為什麼這對 erpilot 戰略很重要

「Hardcode rule = 逼客戶改 code = 不是真 SaaS」

之前 erpilot 寫死「WO release 需 BOM」就跟 SAP / 鼎新一樣硬。v3.25 起改為 PolicyEngine 後：

-  客戶在 UI 開關規矩 → 0 deploy / 0 顧問費
-  AI 對話建規矩 → 小白能客製化
-  Plugin 機制 → 特殊國家 / 行業可擴充
-  Audit log → ISO/GMP/FDA 合規
-  主管覆寫 → 彈性 + 留證

這是 erpilot 真正能打贏鼎新/SAP 的關鍵差異化——他們做不到的「使用者自己改規矩」，erpilot 做到了。

v3.25 (2026-05-18) · erpilot 原創設計 · 全球首見 AI 對話式 ERP rule engine